

Security Review

Cantina Managed

SKY: LZ-INIT-LIB

Cantina Managed review by:

Christoph Michel, Lead Security Researcher

M4rio.eth, Lead Security Researcher



May 20, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Informational	4
3.1.1	<code>activateOft</code> does not verify all route-affecting OFT configuration	4
3.1.2	<code>README</code> Use Cases clarifications	4



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

From May 11th to May 15th the Cantina team conducted a review of [lz-init-lib](#) on commit hash [81ba8961](#). The team identified a total of **2** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	2	1	1
Total	2	1	1

The Cantina Managed team reviewed Sky's [lz-init-lib](#) holistically on commit hash [e2f900fa](#) and concluded that all findings were addressed and no new vulnerabilities were identified.



3 Findings

3.1 Informational

3.1.1 `activateOft` does not verify all route-affecting OFT configuration

Severity: Informational.

Context: `LZInit.sol#L337-L370`.

Description: `_verifyOftConfig` is used before `activateOft` turns on rate limits, so it is acting as the “this route is configured correctly” check. It currently verifies the main LZ/OFT config, but a few route-affecting settings are still outside the check:

- `msgInspector`, which can revert `quoteSend` / `send`.
- Fee config for the `remoteEid`.
- Active receive-library timeout, where an old receive lib can still be accepted during the LZ grace period.

So the activation can pass even though the effective route behavior is not fully pinned.

Recommendation: Add these checks if activation is meant to verify the definitive OFT route config:

```
require(oft_.msgInspector() == cfg.msgInspector, "LZInit/msg-inspector-mismatch");
```

Checking configured fee bps/default fee:

```
(uint16 feeBps, bool feeEnabled) = oft_.feeBps(remoteEid);
require(feeBps == cfg.feeBps, "LZInit/fee-bps-mismatch");
require(feeEnabled == cfg.feeEnabled, "LZInit/fee-enabled-mismatch");
```

Also reject active receive-lib grace periods:

```
(address timeoutLib, uint256 expiry) = ep.receiveLibraryTimeout(oft, remoteEid);
require(timeoutLib == address(0) || expiry <= block.number,
  ↪ "LZInit/recv-timeout-active");
```

If default receive libs are allowed, also account for `defaultReceiveLibraryTimeout(remoteEid)`.

In addition, the following further sanity checks can be considered:

1. Checking the `pausers` against an expected list.
2. For L2s: `SkyOFTAdapterMintBurn` token mint authorization: `oft_.token().wards(oft_) == 1`.
3. `send` / `receive` libs: explicit vs default fallbacks. Check `getReceiveLibrary`'s bool return value and `!isDefaultSendLibrary(oft, remoteEid)`.

Sky: Acknowledged. Some additional checks have been added in commit `e2f900fa`.

Cantina Managed: Acknowledged.

3.1.2 `README` Use Cases clarifications

Severity: Informational.

Context: See each case below.

Description:

1. `README.md#L26-L28`:

Disclaimer: ordering of relayed calls.

LZ does not guarantee the execution order of relayed messages. If a spell relays



more than one message and order matters for safety, bundle the L2-side work into a single message via `LZL2Spell.multiple` and/or split the work across multiple spells.

The disclaimer is only about the execution order. Consider rephrasing the title to a generic disclaimer and extending the existing section to also cover that the following "Use Cases" section shouldn't be taken as definitive recipes/templates and each scenario must be properly analyzed in practice.

2. README.md#L58:

For simplicity we assume the new DVN set is a superset of the old required set.

After Spell 1 execution, there is a brief period when the L2 DVN change message has not been relayed yet, so the L1 is using the new Send DVN config but the L2 is still using the old Receive DVN config. In the other case, when the new DVN set cannot satisfy the old required set, L1 → L2 OFT transfers that happen during this period might be permanently unrecoverable as they can't be verified. Consider using stronger language here to indicate this failure mode, for example, by explicitly saying that this sequence of DVN changes only works when the old set is satisfiable by the new set, otherwise, these steps might lead to permanent fund lockups.

Recommendation: Consider addressing the mentioned topics.

Sky: Addressed in commit [e2f900fa](#).

Cantina Managed: Fix verified.